COMS W4771: Machine Learning

Lecture 20: Neural Networks

Tony Dear, Ph.D.

Department of Computer Science

School of Engineering and Applied Sciences

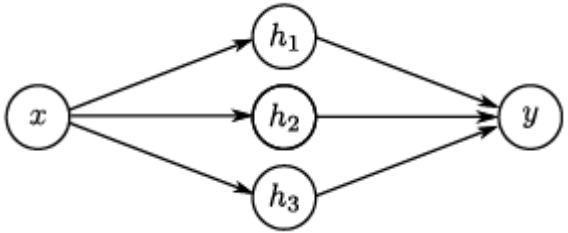
Topics

- Single-layer neural networks
- Universal approximation theorem
- Deep neural networks
- Loss functions

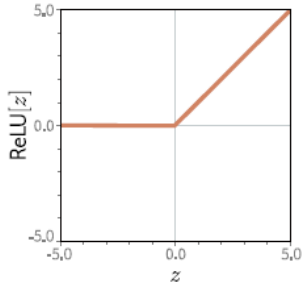
Single-Layer Networks

- Recall the idea of additive models, e.g. for boosting methods
- In a *single-layer*, *feed-forward* **neural network**, the a_d are *activation functions* of linear combinations of the input features:

$$f(x) = \phi_0 + \sum_{d=1}^{D} \phi_d a_d \left(w_{d0} + \sum_{j=1}^{p} w_{dj} x^{(j)} \right)$$



- The activation function is typically nonlinear; otherwise, f is just a composition of linear functions -> still a linear function

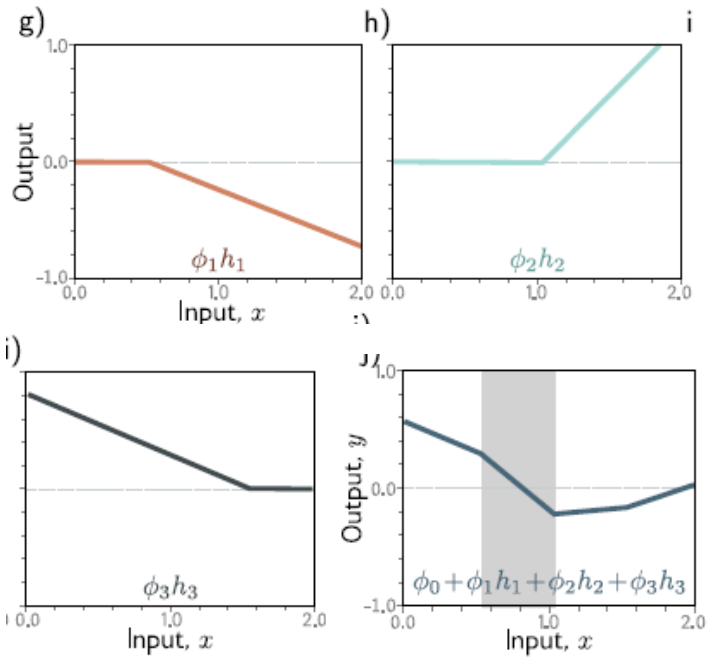


- Example: **Rectified linear unit (ReLU)** $a(z) = ReLU(z) = \max(0, z)$

Prince

Example: ReLU

- A single ReLU function is a piecewise-linear function with two "sections"
- The w_d and ϕ_d parameters shift and scale the ReLU output
- Their sum is a piecewise-linear function with up to $D + 1$ separate "sections"
- With more enough units, we can approximate an arbitrarily complex function!



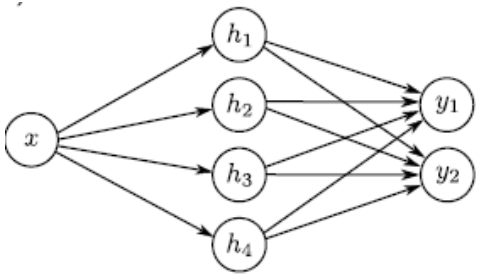
Prince

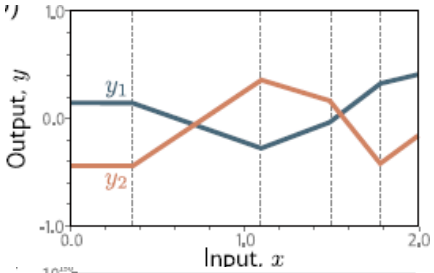
Higher Dimensions

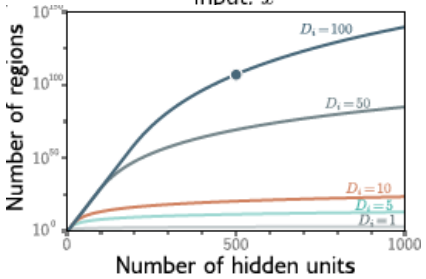
- It is also possible to have multivariate outputs y
- Each one has its own set of weights ϕ_{kd}, but their output functions share the same "joints"

- When x is multidimensional, each individual ReLU outputs a piecewise-linear surface
- Combine into convex polygonal or polytope regions

- If $x \in \mathbb{R}^p$ and we have p ReLU units, then we can learn a function with up to 2^p separate regions







Prince

Universal Approximation Theorem

- Consider a single-layer neural network with nonlinear activation functions

- **Universal approximation theorem**: This can approximate *any* continuous function on a compact domain (closed and bounded subset of $\mathbb{R}^n$)
- Moreover, the approximation can be made *arbitrarily accurate* with sufficiently many activation units

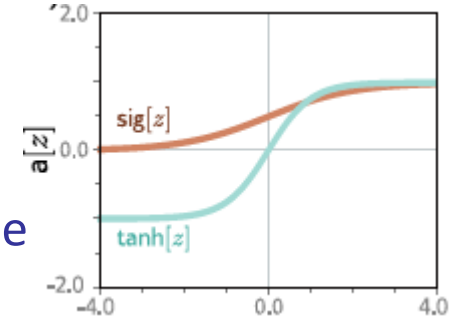
- The result is *existential*: A function can be represented, but no guarantees on how many units we need, or that we can actually find its approximation
- This also does not say anything about recovering the function *exactly*

Other Activation Functions

- Prior to the 2010s, sigmoid and tanh were used more often than ReLU
- Advantages of ReLU: Easy to compute, derivative is 1 in the positive region
- Derivatives of sigmoid and tanh both go to 0 as $z \rightarrow \infty$

- Disadvantage of ReLU is that its derivative is 0 in the negative region
- Leads to little or no updates in gradient descent

- ReLU is generally good enough, but some variants (leaky ReLU, softplus, swish) can sometimes improve performance



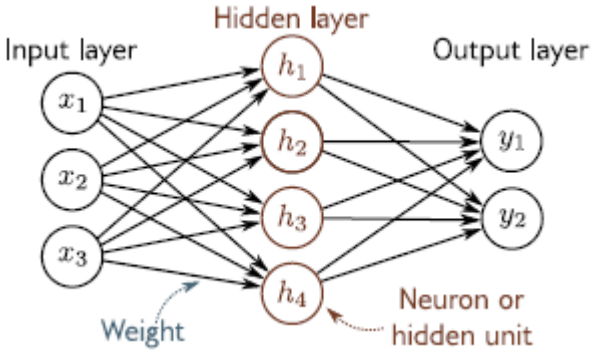
Prince

Neural Network Terminology

- A general neural network has an *input layer, output layer,* and one or more *hidden layers* containing *hidden units*, which output *activations*
- Neural networks are also called **multi-layer perceptrons (MLPs)**

- Networks with one hidden layer are *shallow*; otherwise, they are *deep*
- If every node in one layer connects to every node in the next layer, then the network is *fully connected*

- The edges represent *network weights*, and intercept parameters are also called *biases*

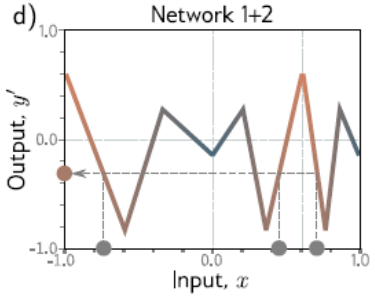


Prince

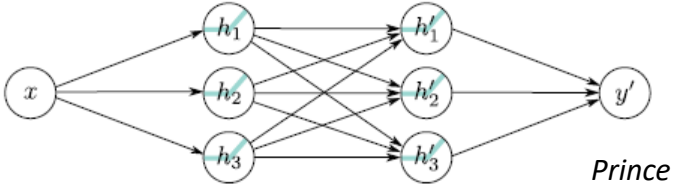
Composing Neural Networks

- We can *compose* two or more neural networks to describe a larger class of functions with the same number of parameters

- Ex: With 2 hidden layers, each h_i splits feature space into 2 regions, and each h_i' splits into $2 \times 2^3 = 16$ regions
- With 1 hidden layer of 6 units, we can only get 7 regions!



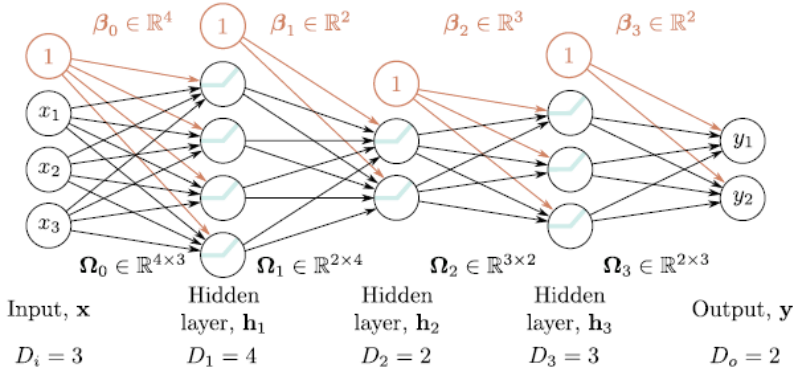
- The *width* and *depth* of the networks are hyperparameters, typically chosen before training the weights



Prince

Deep Neural Networks

- Suppose we have K hidden layers
- Layer k has D_k units, input D_i, output D_o

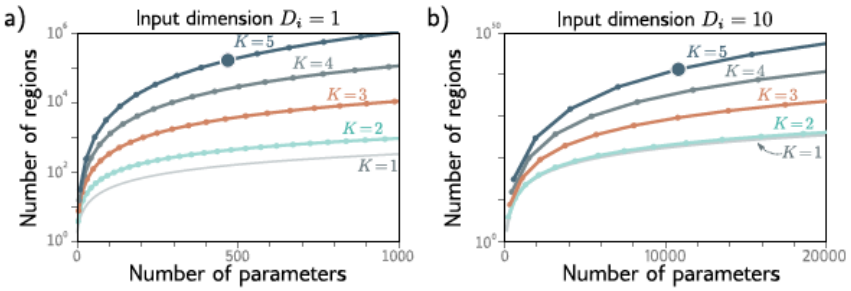


- Parameters for $0 \leq k \leq K$: weights matrix $\boldsymbol{\Omega}_k \in \mathbb{R}^{D_{k+1}\times D_k}$, bias vector $\beta_k \in \mathbb{R}^{D_{k+1}}$

- Each layer computes the activation vector

$$\mathbf{h}_k = \mathbf{a}(\boldsymbol{\beta}_{k-1} + \boldsymbol{\Omega}_{k-1}\mathbf{h}_{k-1})$$

- Representation power grows rapidly with number of hidden layers!



Prince

Architecture Design

- Each neural network layer allows for additional *interaction* between the (transformed) features, giving us new *representations* of them

- Deeper networks allow for many composition and transformation steps
- Wider networks provide more capacity for capturing information
- Too small may lead to underfitting; too large may lead to overfitting

- More common to start with commonly used architectures for different tasks
- **Transfer learning**: Start with a *general* pretrained model (e.g., open-source) and further fine-tune on curated data for specialized tasks

Classification

- A single output unit y with identity activation is suitable for regression tasks
- $y = \boldsymbol{\beta}_K + \boldsymbol{\Omega}_K \mathbf{h}_K$ is simply a linear model applied to the final hidden layer

- For binary classification, we can encode our class as $y \in \{0,1\}$ and apply a *sigmoid* activation to transform the output into a probability

- For multiclass classification with $D_o > 2$ classes, we have D_o output units and apply a *softmax* activation to output a valid probability distribution:

$$a(z_l) = \Pr(y = l|\mathbf{x}) = \frac{\exp(z_l)}{\sum_{m=1}^{D_0} \exp(z_m)}$$

Loss Functions

- We will need loss functions to compare neural network predictions to sample outputs; as usual, we will use squared error (or variants) for regression

- For classification, encode $\mathbf{y} \in \{0,1\}^{D_o}$ as a 1-hot vector where $y_k = 1$ iff $y = k$
- Then we can compactly write the likelihood $\Pr(y|\mathbf{x}) = \prod_{l=1}^{D_o} a(z_l)^{y_l}$
- The negative log-likelihood is then

$$NLL = -\sum_{l=1}^{D_o} y_l \log \frac{\exp(z_l)}{\sum_{m=1}^{D_0} \exp(z_m)} = -\sum_{l=1}^{D_o} y_l z_l + \sum_{l=1}^{D_o} y_l \log \sum_{m=1}^{D_0} \exp(z_m)$$

- Since only $y_k = 1$, and $\sum_{l=1}^{D_o} y_l = 1$, $NLL = -z_k + \log \sum_{m=1}^{D_0} \exp(z_m)$

Cross-Entropy Loss

- The NLL can also be derived as the *cross-entropy* between the true "distribution" $\mathbf{y}$ and softmax predictions $\hat{\mathbf{y}}$
- **Kullback-Leibler divergence** between from $\mathbf{y}$ to $\hat{\mathbf{y}}$**:**

$$D_{KL}(\mathbf{y}||\hat{\mathbf{y}}) = \sum_{l=1}^{D_o} y_l \log \frac{y_l}{\hat{y}_l} = \sum_{l=1}^{D_o} y_l \log y_l - \sum_{l=1}^{D_o} y_l \log \hat{y}_l$$

- We want to *minimize* D_{KL} over predictions $\hat{y}_l$, so we can drop the first term

$$\operatorname{argmin}_{\phi} D_{KL}(\mathbf{y}||\hat{\mathbf{y}}) = \operatorname{argmin}_{\phi} \left(-\sum_{l=1}^{D_o} y_l \log \hat{y}_l \right)$$

- Remaining second term is the **cross-entropy** and is a valid objective even for a general target distribution $\mathbf{y}$

Summary

- Neural networks apply nonlinear activation functions to linear combinations of inputs to approximate functions

- More units and/or more layers can increase representative ability
- Universal approximation theorem states that approximations can be made arbitrarily accurate for a broad class of functions

- Lots of considerations in architecture design and training
- For classification, apply a softmax output leading to cross-entropy loss